

Phase 1-5. JWT 인증 (Access + Refresh + Rotation) 학습정리

이 문서의 목적 — 며칠에 걸쳐 만든 1-E JWT 인증을 처음부터 끝까지 복습한다. 자고 일어나면 잊어버리는 걸 막기 위해, 용어 하나하나를 풀어서 설명하고, 남들이 많이 하는 질문·실수를 함께 정리했다. 코드보다 "왜 그렇게 했는가"에 집중.

0. 큰 그림 — 우리가 만든 것

로그인 시스템을 만들었다. 단순한 로그인이지 아니라 **실무급 인증**이다:

```
회원가입 □ 로그인 □ (Access 만료되면) 갱신 □ 로그아웃  
/register /login /refresh /logout
```

이 네 개의 엔드포인트(API 주소)가 전부 실제로 동작한다. 그리고 그 안에 Access/Refresh 토큰 분리, httpOnly 쿠키, Token Rotation, 재사용 탐지까지 들어있다.

왜 이렇게까지 하나? — 아래에서 용어를 하나씩 풀며 이유를 설명한다.

1. 근본 문제 — HTTP는 기억을 못 한다

용어: stateless (무상태)

stateless = "상태가 없다" = "이전 요청을 기억 못 한다"

HTTP(웹 통신 규약)는 요청 하나하나가 독립적이다. 서버는 방금 전 요청을 기억하지 않는다. 그래서 로그인을 한 번 해도, 다음 요청에서 서버는 "당신 누구세요?"를 다시 모른다.

- 현실 비유: 매번 처음 보는 사람처럼 대하는 은행 창구. 방금 신분증을 보여줬어도, 다음 창구에선 또 보여줘야 한다.
- 그래서 매 요청마다 "나 로그인한 사람이야"를 증명해야 한다. 그 증명 수단이 **토큰**이다.

용어: 토큰 (token)

토큰 = "출입증". 로그인에 성공하면 서버가 발급하는 증명서. 이후 요청마다 이 토큰을 들고 가면 "아, 인증된 사람이구나"라고 서버가 알아준다.

2. JWT — 서버가 저장 안 해도 검증되는 출입증

용어: JWT (JSON Web Token)

우리가 쓴 토큰의 종류. "제이슨 웹 토큰"이라고 읽는다. 세 부분으로 구성된다:

```
헤더.페이로드.서명  
xxxxx.yyyyy.zzzzz ← 실제로는 이렇게 점(.)으로 구분된 긴 문자열
```

- 헤더(header)** — 토큰 종류와 서명 알고리즘 정보 (예: "HS256으로 서명했음")
- 페이로드(payload)** — 실제 담긴 정보. 아래 "클레임" 참고
- 서명(signature)** — 위조 방지용 도장. 서버만 아는 비밀키로 만든다

JWT는 항상 **eyJ** 로 시작한다. 헤더 부분이 { " 로 시작하는데, 이걸 Base64로 인코딩하면 **eyJ** 가 되기 때문. 로그인 응답에서 **eyJhbG...** 가 나오면 "아 JWT구나" 하면 된다.

용어: 클레임 (claim)

클레임 = "토큰에 담긴 정보 조각". 페이로드 안에 들어간다. 우리가 담은 것:

- **sub** (subject) — 토큰의 주인. UserId를 넣었다. JWT 표준 이름
- **tenantId** — 어느 테넌트(정비소) 소속인가. ★ 멀티테넌시의 핵심 열쇠
- **email** — 이메일
- **jti** (JWT ID) — 토큰마다 고유한 ID. 매번 다르게 생성

용어: 서명 (signature) 과 HS256

서명 = "이 토큰이 위조되지 않았다는 도장"

서버는 자기만 아는 **비밀키(secret key)**로 토큰에 서명한다. 누군가 페이로드를 조작하면(예: 남의 tenantId로 바꿔 남의 데이터를 보려 하면) 서명이 안 맞아서 서버가 "위조!"라고 걸러낸다. 비밀키가 없으면 유효한 서명을 못 만든다.

- **HS256** = 서명에 쓴 알고리즘 이름. "HMAC-SHA256"의 줄임말. **대칭키 방식** — 발급할 때랑 검증할 때 **같은 비밀키**를 쓴다. (반대는 비대칭키. 발급/검증 키가 다름. 우리는 단순한 대칭키로 충분)

핵심 장점과 치명적 단점

장점: 서버가 토큰을 DB에 저장 안 해도, **서명만 확인하면** "내가 발급한 진짜"인지 알 수 있다. DB 조회가 없으니 빠르고 확장성이 좋다. (이게 stateless의 장점)

단점: 저장을 안 하니까 **한번 발급하면 만료 전까지 취소가 불가능하다**. - 로그아웃해도 그 JWT는 만료까지 유효 - 탈취당하면 만료까지 계속 악용됨 - "이 사용자 차단!"이 안 됨

이 단점이 다음 설계(Access + Refresh 분리)의 출발점이다.

3. Access + Refresh 토큰 분리 — 딜레마의 해법

딜레마

토큰 수명(유효 기간)을 정해야 하는데: - **길게** 하면 → 편하다(재로그인 안 함). 근데 탈취 시 오래 위험 - **짧게** 하면 → 안전하다. 근데 자주 만료돼서 불편 (계속 재로그인)

둘 다 잡으려고 **토큰을 두 개로 나눈다**. 업계 표준이다.

용어: Access Token (접근 토큰)

- 실제 API 요청마다 들고 다니는 출입증
- **수명 짧음 (우리는 15분)**. 탈취해도 금방 만료 → 피해 작음
- 서버에 저장 안 함 (stateless). 서명만 확인
- 저장 위치: 브라우저 **메모리** (JS 변수)

용어: Refresh Token (갱신 토큰)

- Access가 만료되면 이걸로 **새 Access를 발급**받음. "출입증 갱신권"
- **수명 김 (우리는 14일)**. 이게 있으면 로그인 유지됨
- **서버(DB)에 저장** ← 핵심. 그래서 무효화가 가능(로그아웃 시 삭제/폐기)
- 저장 위치: **httpOnly 쿠키** (아래 설명)

왜 이 비대칭이 정답인가

짧게 하면		길게 하면
Access	안전 (자주 쓰이니 위험 노출 큼)	위험
Refresh	재로그인 잦음 (불편)	로그인 유지 (목적 달성)

- **Access는 짧아야** 안전하고, **Refresh는 길어야** 편하다
- "자주 쓰는 건 짧고 안전하게, 갱신권은 길지만 서버가 통제"

[흔한 질문] Refresh도 짧게 하면 안 되나? → 안 된다. Refresh의 목적이 "로그인 유지"인데, 짧게 하면 자주 만료돼서 재로그인을 계속 해야 한다. 그러면 Refresh를 만든 이유가 사라진다. 대신 긴 Refresh의 위험은 아래 Rotation + 재사용 탐지로 관리한다.

중요: Access는 JWT, Refresh는 그냥 랜덤 문자열

- **Access** = JWT (서명된 의미 있는 토큰). 서버가 저장 안 하고 서명으로 검증하니 JWT가 맞다
- **Refresh** = 그냥 **랜덤 문자열** (JWT 아님!). 어차피 DB에 저장하니 서명이 필요 없다. "추측 불가능한 랜덤"이면 충분하고, DB 조회로 검증한다

[흔한 오해] Refresh Token이 GUID인가? → 아니다. 우리는 **512비트(64바이트) 암호학적 난수**를 Base64 문자열로 만들었다. GUID(128비트)보다 훨씬 크다. GUID는 엔티티의 Id(기본키)에 쓰고, Refresh 토큰 값은 별개다.

4. httpOnly 쿠키 — Refresh를 안전하게 보관

용어: 쿠키 (cookie)

쿠키 = 브라우저가 저장하는 작은 데이터 조각. 서버가 "이거 저장해줘"라고 지시하면 브라우저가 보관하고, 이후 그 서버로 요청 보낼 때마다 자동으로 함께 보낸다.

용어: httpOnly

httpOnly = "자바스크립트는 이 쿠키를 못 읽는다"는 표시

- 일반 쿠키/localStorage는 JS가 `document.cookie`, `localStorage.getItem()` 으로 읽을 수 있다
- 근데 httpOnly 쿠키는 **JS가 접근 자체를 못 한다**. 존재조차 안 보인다
- **왜 중요한가**: XSS 공격(아래) 방어. 악성 스크립트가 페이지에 심어져도 Refresh Token을 훔칠 수 없다. JS한테 안 보이니까

용어: XSS (Cross-Site Scripting)

XSS = 악성 스크립트를 페이지에 몰래 심는 공격. 예를 들어 게시판에 `<script>` 를 심어서, 다른 사용자가 그 글을 볼 때 스크립트가 실행되게 하는 것. 이 스크립트가 `localStorage` 나 일반 쿠키의 토큰을 훔쳐간다. httpOnly면 못 훔친다.

httpOnly가 실제로 작동하는 방식 (중요)

JS (자바스크립트) 못 읽음 (숨겨짐)	브라우저 저장·자동전송 요청 시 자동 첨부	서버 Set-Cookie로 "저장해" 지시 Request.Cookies로 읽음
---------------------------	-------------------------------	---

- 서버가 응답에 **Set-Cookie** 헤더를 넣어 "이거 저장해"라고 지시 (우리 코드의 `Response.Cookies.Append(...)` 가 이 헤더를 만든다)
- 브라우저가 저장. httpOnly라 JS엔 안 보이게
- 이후 요청마다 브라우저가 자동으로 쿠키를 실어 보냄
- 서버는 `Request.Cookies["refreshToken"]` 로 읽음

핵심: "JS는 못 읽지만, 브라우저는 요청 시 자동으로 서버에 보낸다." 그래서 갱신할 때 클라이언트가 Refresh를 직접 안 보내도, 브라우저가 쿠키로 보내주고 서버가 읽는다.

용어: Secure, SameSite (쿠키 옵션)

우리가 쿠키에 붙인 옵션들: - **HttpOnly = true** — 위 설명. JS 접근 차단 - **Secure = true** — "HTTPS 연결에서만 이 쿠키를 보낸다." HTTP(암호화 안 됨)로는 안 보내 중간 가로채기 방지. 로컬 테스트가 HTTP면 쿠키가 안 심어지는 함정 (아래 실수 참고) - **SameSite = Strict** — CSRF 방어. "다른 사이트에서 온 요청엔 이 쿠키를 안 보낸다"

용어: CSRF (Cross-Site Request Forgery)

CSRF = 다른 사이트에서 몰래 요청을 보내는 공격. 예: 악성 사이트가 사용자 몰래 "내 은행 계좌로 송금" 요청을 사용자의 쿠키와 함께 보내게 하는 것. SameSite=Strict면 다른 사이트발 요청엔 쿠키를 안 실어서 막는다.

Todo(02)와의 차이 (면접 카드)

- **02:** JWT 하나를 localStorage에 저장 (단순, XSS 취약)
- **03:** Access(메모리) + Refresh(httpOnly 쿠키) 분리 (구글/네이버 방식)
- 면접 답변: "왜 localStorage가 아니라 httpOnly 쿠키인가" → XSS 방어 + 토큰 분리

5. Token Rotation + 재사용 탐지 — 긴 Refresh의 위험 관리

용어: Token Rotation (토큰 회전)

Rotation = "Refresh를 쓸 때마다 폐기하고 새로 발급" = 일회용으로 만들기

로그인	→ Refresh#1 발급	
갱신	→ Refresh#1 폐기 + Refresh#2 발급	← 한 번 쓰면 버림
갱신	→ Refresh#2 폐기 + Refresh#3 발급	

한 번 쓴 Refresh는 두 번 못 쓴다. "회전(rotation)"이라 부르는 이유.

용어: 재사용 탐지 (reuse detection)

재사용 탐지 = "이미 폐기된 토큰이 다시 오면 탈취로 간주"

정상 사용자: Refresh#1 사용 → Refresh#2 받음 (Refresh#1 폐기됨)
공격자가 훔친 Refresh#1로 갱신 시도
→ 서버: "어? Refresh#1은 이미 폐기됐는데 또 왔네?"
→ 정상 사용자라면 폐기된 걸 다시 쓸 리 없음 = 탈취 신호!
→ 그 유저의 모든 토큰 무효화 → 공격자·정상유저 다 재로그인

[흔한 질문] 재사용 탐지가 토큰 중복이랑 관련 있나? → 아니다. 재사용 탐지는 "같은 토큰이 폐기 후 다시 등장"을 잡는 것(IsRevoked로 판단). 토큰 중복(서로 다른 발급인데 값이 우연히 겹침)이랑은 완전히 별개다. 512비트 난수는 중복 확률이 사실상 0이고, 만에 하나도 DB 기본키/유니크 제약이 막는다.

용어: soft delete (소프트 삭제)

soft delete = "실제로 지우지 않고, '삭제됨' 표시만 하는 것"

우리는 토큰을 폐기할 때 DB에서 진짜 지우지 않고 **IsRevoked = true** 로 **표시만** 한다. - **왜 안 지우나:** 재사용 탐지 때문. 폐기된 토큰이 남아있어야 "폐기된 게 또 왔다=탈취"를 감지할 수 있다. 지우면 탈취인지 그냥 잘못된 토큰인지 구분 못 함 - **부작용:** 토큰이 계속 누적된다 → 나중에 cleanup(정리) 로직 필요 (TODO)

[TODO] Refresh Token 누적 문제 — 폐기된 토큰이 soft delete로 계속 쌓인다. 무한 누적을 막으려면 "만료된 지 오래된 토큰 주기적 삭제" 같은 cleanup이 필요. Phase 2나 1-E 마무리 때 구현 예정. (면접: soft delete로 탐지 유지 + 누적 방지 트레이드오프)

6. 클린 아키텍처 — 코드를 어디에 두었나

이번에 만든 것들을 계층별로 나눴다. 이게 이번 학습의 핵심 중 하나다.

원칙: 규칙은 안쪽, 구현은 바깥쪽

API (Presentation)	← HTTP 받기, 쿠키 설정, 컨트롤러
Application	← 업무 규칙 (인증 흐름). DB를 모른다!
Infrastructure	← 실제 기술 (EF Core, DB, 해싱)
Domain	← 엔티티 (아무것도 참조 안 함)

핵심 감각: Application은 DB를 모른다

- **Application** = "무엇을 할지"(규칙). `AuthService` 를 열면 EF Core/SQL이 한 줄도 없다
- **Infrastructure** = "어떻게 할지"(기술). `UserRepository` 를 열어야 `m_Db.Users.Add()` 가 나온다
- **판단 기준**: "이게 DB 냄새(EF Core, SQL)가 나나?" → 나면 Infrastructure

왜? — 나중에 PostgreSQL → MSSQL로 바뀌어도 Application은 한 줄도 안 바뀐다. Infrastructure의 구현만 갈아끼우면 된다. (레거시→모던 전환 시 진가 발휘)

용어: 인터페이스 (interface)

인터페이스 = "이런 기능을 제공하겠다는 약속(계약)". 실제 코드(어떻게)는 없고, "무엇을 할 수 있는지"만 정의. 이름 앞에 **I** 를 붙이는 관례 (`IUserRepository` 등)

Application:	<code>IPasswordHasher</code> (약속 — "해싱할 수 있다")
	↑ 구현
Infrastructure:	<code>PasswordHasher</code> (실제 구현 — PBKDF2로 해싱)

용어: 의존성 역전 (Dependency Inversion)

의존성 역전 = "안쪽(Application)이 바깥(Infrastructure)의 구체적 구현이 아니라, 추상(인터페이스)에만 의존하게 만드는 것"

- `AuthService`는 `IPasswordHasher` (약속)만 안다. PBKDF2인지 bcrypt인지 모른다
- 나중에 해싱 방식을 바꾸려면 Infrastructure 구현만 교체. Application은 안 건드림

용어: DI (Dependency Injection, 의존성 주입)

DI = "필요한 부품을 밖에서 만들어 넣어주는 것"

- `AuthService`가 `IPasswordHasher` 를 직접 만들지 않고, **생성자로 받는다**
- 누가 넣어주나? → DI 컨테이너. `Program.cs` 에서 등록: `builder.Services.AddScoped<IPasswordHasher, PasswordHasher>();` = "누가 `IPasswordHasher`를 요구하면 `PasswordHasher`를 만들어 줘라"

용어: Composition Root (조립 지점)

Composition Root = "인터페이스와 구현을 실제로 이어붙이는 곳" = 우리의 `Program.cs`

- API가 Infrastructure를 참조하는 건 위반이 아니다. "조립"은 진입점의 책임이니까
- 여기서 모든 부품(해서, 토큰생성기, 리포지토리, 서비스)을 등록한다

용어: DI 수명 (lifetime) — Scoped / Singleton / Transient

`AddScoped` 의 그 "Scoped"가 수명이다: - **Singleton** — 앱 전체에 하나. 상태 없는 전역 도구 - **Scoped** — 요청 하나당 하나. 대부분의 서비스 (우리 가 쓴 것) - **Transient** — 요청할 때마다 새로

우리는 인증 서비스들을 다 Scoped로 통일. (`DbContext`도 Scoped)

7. Repository 패턴 — DB 접근을 감싸기

용어: Repository (저장소) 패턴

Repository = "데이터 저장소를 감싸는 창고 관리인". "어떻게 저장/조회하는지(DB 세부)"를 감추고 "저장한다/찾는다"는 의도만 노출.

```
Repository 없이: m_Db.Users.Add(user); await m_Db.SaveChangesAsync(); ☐ DB 세부 노출  
Repository 있어: await m_UserRepository.AddAsync(user); ☐ 의도만
```

왜 IAuthService에 다 안 하고 Repository를 따로 두나

- **의존성 방향**: AuthService(Application)가 DbContext(Infrastructure)를 직접 쓰면 방향이 거꾸로 됨(위반). Repository 인터페이스를 Application에 두면 방향이 올바름
- **역할 분리**: 인증 로직(정책)과 데이터 접근(기술)을 분리 → 각자 한 가지만 책임

실용적으로 — 필요한 것만

모든 엔티티에 Repository를 만들지 않았다. 인증에 필요한 User, RefreshToken만.

면접 카드: "Repository 패턴을 기계적으로 모든 엔티티에 적용하지 않고, 계층 분리가 필요한 곳에만 뒀다. 단순 CRUD는 가볍게. 패턴을 아는 것보다 언제 쓸지 판단하는 게 중요."

8. EF Core 핵심 개념

용어: EF Core (Entity Framework Core)

EF Core = .NET의 ORM. ORM(Object-Relational Mapping) = "객체(C# 클래스)와 관계형 DB(테이블)를 자동으로 연결해주는 도구." SQL을 직접 안 쓰고 C# 코드로 DB를 다룬다.

용어: 변경 추적 (Change Tracking)

변경 추적 = "EF Core가 가져온 엔티티의 변화를 감시하다가, SaveChanges 때 한 번에 반영"

```
var tokens = await m_Db.RefreshTokens.Where(...).ToListAsync(ct); // 가져오면서 추적 시작  
foreach (var token in tokens) token.Revoke(); // 변경 ☐ EF가 "수정됨" 기록 (아직 DB 안 감)  
await m_Db.SaveChangesAsync(ct); // 그동안 쌓인 모든 변경을 한 번에 DB로 (UPDATE 여러 개)
```

- **장바구니 비유**: Revoke()는 "담기(변경 예약)", SaveChanges는 "결제(실제 반영)"
- **장점 1 — 효율**: 여러 변경을 한 번의 DB 왕복으로 처리
- **장점 2 — 원자성**: SaveChanges는 하나의 트랜잭션. 중간에 실패하면 전부 롤백 ("다 되거나 다 안 되거나"). 재사용 탐지의 "유저 토큰 전부 무효화"에 딱 맞음

용어: 마이그레이션 (Migration)

마이그레이션 = "C# 엔티티의 변화를 DB 스키마(테이블 구조) 변경으로 옮기는 것"

```
dotnet ef migrations add AddRefreshToken ← 변경 설계도(코드) 생성  
dotnet ef database update ← 실제 DB에 반영 (CREATE TABLE 등)
```

- 첫 마이그레이션(InitialCreate)은 전체 테이블 생성, 이후는 **변경분만** 담긴다
- `EFMigrationsHistory` 테이블 = "어떤 마이그레이션이 적용됐나" 출석부

용어: xmin (낙관적 동시성 토큰)

xmin = PostgreSQL의 시스템 컬럼. 행이 수정될 때마다 값이 바뀐다. "동시 수정 충돌"을 감지하는 데 쓴다. MSSQL의 rowversion 과 같은 개념(이름만 다름). - 이번엔 직접 안 썼지만 BaseEntity에 이미 [Timestamp] uint Version 으로 있음 - Phase 2 예약 CRUD에서 본격 사용 예정

용어: CancellationToken (취소 토큰)

CancellationToken = "진행 중인 작업을 중간에 취소할 수 있게 하는 신호"

- 사용자가 요청 후 브라우저를 닫으면 → 서버가 불필요한 DB 작업을 멈춤
- 어디에 넣나: 비동기(async)이고 I/O를 기다리는 메서드에. 즉시 끝나는 동기 메서드엔 X
- 핵심: 받기만 하면 무의미. 받아서 **안쪽으로 계속 전달**해야 작동 (Controller → Service → Repository → SaveChangesAsync(ct))

9. C# 문법 용어 정리

며칠간 나온 문법들. 자고 일어나면 헛갈리는 것들이라 모아둔다.

? — nullable (null 가능)

Type? = "이 값이 null일 수 있다"는 표시

```
User    → 반드시 User 객체가 있음 (null 아님)
User?   → User가 있거나, 없을 수도(null) 있음
```

- "찾기(Find)" 메서드는 대부분 ? 가 붙는다. 못 찾으면 null을 돌려주니까
- ? 가 붙으면 컴파일러가 "쓰기 전에 null 확인해!"라고 강제 → 널 참조 예외 예방

! — null-forgiving (null 아님 보장)

값! = "이건 null 아니야, 내가 보장해" (컴파일러 경고 무시)

- m_Config["Jwt:SecretKey"]! — "설정에 값 넣었으니 null 아님"
- 우기는 것이라, 더 안전한 건 ?? throw (값 없으면 명확한 예외). fail-fast 방식

is null / is not null

- x is null — x가 null이면 true
- x is not null — x가 null이 아니면 true (!= null 과 같지만 더 읽기 쉬움)

[내가 한 실수] 로그아웃에서 is not null 을 is null 로 잘못 써서, 조건이 뒤집혀 Revoke()가 실행 안 됐다. "메시지는 났는데 DB는 안 바뀐" 미스터리 의 범인. → 응답 코드만 믿지 말고 실제 DB를 확인하는 습관으로 잡아냄

record

record = "간결한 불변 데이터 클래스" (C# 9부터, 2020년)

```
public record LoginUserRequest(string Email, string Password);
```

- 괄호 안 파라미터가 **자동으로 읽기전용 속성**이 된다 → request.Email 로 접근
- DTO(데이터 전송용)에 class보다 record가 요즘 관례

익명 객체 new { }

익명 객체 = "이름 없는 임시 객체를 즉석에서 만들기" (C# 3.0부터, 2007년)

```
return Ok(new { message = "로그아웃되었습니다." });
```

- 클래스 정의 없이 그 자리에서 객체 생성 → JSON { "message": "..." } 로 변환됨
- 일회성 간단 응답에 사용. 재사용/명확한 타입 필요하면 record

file-scoped namespace

namespace X; (세미콜론) — C# 10부터(2021년). 종괄호 없이 네임스페이스 선언. 파일 전체가 그 네임스페이스에 속함. 들여쓰기가 줄어 깔끔.

[버전 감각 — 면접 포인트] 익명 객체(C# 3.0)는 어디서든 되지만, record(C# 9), file-scoped namespace(C# 10)는 최신 .NET에서만. 레거시(.NET Framework)↔모던(.NET 8+)을 오갈 때 "어느 문법이 어느 버전부터인지" 아는 게 실무 감각.

10. 비밀번호 해싱 개념

용어: 해시 (hash)

해시 = "원본을 되돌릴 수 없는 단방향 변환값"

회원가입: "mypassword123" → [해싱] → "AQAAAAIAAYag..." → DB 저장
로그인: "mypassword123" → [해싱] → "AQAAAAIAAYag..." → DB값과 비교 → 일치!

- 단방향: 평문→해시는 되는데, 해시→평문은 안 된다. DB 털려도 원본 비밀번호 못 알아냄
- 같은 입력=같은 출력: 그래서 로그인 시 비교 가능

용어: 솔트 (salt) 와 반복

단순 해시는 취약해서 두 가지를 더한다: - 솔트(salt) = "각 비번에 섞는 랜덤 값". 같은 비번도 사람마다 다른 해시가 됨 → 레인보우 테이블(미리 계산된 해시표) 공격 무력화 - 반복 = 해시를 수만 번 반복해 일부러 느리게. 무차별 대입 공격 방어

용어: PBKDF2

우리가 쓴 해싱 알고리즘. ASP.NET Core의 `PasswordHasher` 가 PBKDF2 + 솔트 + 반복을 다 해준다. 직접 구현 안 하고 검증된 걸 가져다 씀.

[흔한 실수] `PasswordHasher`의 `T`가 뭐가? → 원래 사용자 타입을 받게 설계됐지만, 우리는 해싱만 쓰니 아무 타입(`object`)이나 더미로 넣으면 됨. 해싱 자체엔 그 타입이 안 쓰인다.

11. HTTP 상태 코드 (응답 코드)

우리가 쓴 것들: - 200 OK — 성공 - 400 Bad Request — 잘못된 요청 (예: tenantId 비었음 — 리치 도메인 검증 실패) - 401 Unauthorized — 인증 실패 (로그인 실패, 토큰 없음/무효) - 409 Conflict — 충돌 (예: 이미 있는 이메일로 회원가입)

[보안 디테일] 로그인 실패 메시지 "이메일이 없음"과 "비번이 틀림"을 구분하지 않고 "이메일 또는 비밀번호가 올바르지 않습니다"로 뭉뚱그린다. 구분하면 공격자가 "어떤 이메일이 가입돼 있는지" 알아낼 수 있기 때문(계정 열거 공격). 보안 기본기.

12. Scalar — API 테스트 도구

용어: OpenAPI / Swagger / Scalar

- OpenAPI = API를 기술하는 표준 문서 형식 (원래 이름이 Swagger였음)
- Swagger UI = 그 문서를 브라우저에서 보고 테스트하는 화면 (전통적)
- Scalar = Swagger UI의 현대적 대체. OpenAPI 표준은 그대로 계승, 화면만 새것

핵심: Scalar는 "Swagger의 알맹이(OpenAPI)는 계승하고 화면만 현대화한 것." 밑에 깔린 문서(JSON)는 동일. .NET 9부터 마이크로소프트 기본이 이 방향으로 바뀜.

13. 흔한 실수 총정리 (내가 실제로 겪은 것들)

며칠간 실제로 부딪힌 함정들. 다음에 또 안 겪게 정리.

① 오타 — DbSet 이름 = 테이블 이름

`RefreshToken` 을 `Refresh` (r빠짐), `Tokens` 를 `Toekens` (e,k 순서 바뀜)로 오타. → **DbSet 속성명이 그대로 테이블명이 된다.** 마이그레이션 파일을 눈으로 확인해서 잡음. → 교훈: 마이그레이션 생성 후 **반드시 파일 열어서 테이블명·컬럼 확인**

② 이름 충돌 — RegisterRequest

`Microsoft.AspNetCore.Identity.Data` 에 이미 `RegisterRequest`, `LoginRequest` 가 있다. 우리가 같은 이름을 쓰니 엉뚱한 게 잡혀서 "TenantId 정의 없음" 에러. → **해결:** `RegisterUserRequest` 처럼 구별되는 이름 사용. Identity 패키지의 흔한 이름 피하기

③ AddControllers / MapControllers 누락

Phase 0에서 최소 API(`app.MapGet`)로 만들어서, 컨트롤러 등록 코드가 없었다. → 컨트롤러가 인식이 안 됨(엔드포인트 안 보임).

`AddControllers()` + `MapControllers()` 추가로 해결

④ AddAuthorization 누락

`app.UseAuthorization()` 을 넣었는데 짝인 `builder.Services.AddAuthorization()` 이 없어서 에러. → **ASP.NET Core는 Add(등록) ↔ Use(사용)가 짝.** Use 넣으면 대응하는 Add도 필요

⑤ EF Core using 누락

`FirstOrDefaultAsync`, `ToListAsync` 가 인식 안 됨(이상한 오버로드가 잡힘). → `using Microsoft.EntityFrameworkCore;` 필요. 확장 메서드는 using이 있어야 인식됨

⑥ is null vs is not null

로그아웃 조건에서 `is not null` 을 `is null` 로 잘못 씀. 조건이 뒤집혀 `Revoke()` 안 됨. → "성공 응답 ≠ 실제 동작." DB를 직접 확인해서 잡음

⑦ Secure=true + HTTP 로컬 테스트

쿠키의 `Secure=true` 는 HTTPS에서만 전송. 로컬이 HTTP면 쿠키가 안 심어짐. → https 프로파일을 `launchSettings`에 추가하고 `--launch-profile https` 로 실행

⑧ launchSettings에 https 프로파일 없음

Phase 0에서 http만 만들어서 `--launch-profile https` 가 실패. → `launchSettings.json`에 https 프로파일 수동 추가. `dotnet dev-certs https --trust` 도 필요할 수 있음

⑨ 명령어 오타

`migrations` (복수)를 `migration` 으로, `Add-Migration` (패키지관리자콘솔 전용)을 일반 터미널에서 실행. → dotnet CLI는 `dotnet ef migrations add`, 패키지관리자콘솔은 `Add-Migration`. 섞지 말 것

14. 전체 흐름 복습 — 4개 엔드포인트

회원가입 (/api/v1/auth/register)

이메일·비번·테넌트ID 받기
→ 이미 있는 이메일인가 확인 (IUserRepository)
→ 비번 해싱 (IPasswordHasher.Hash)
→ User 엔티티 생성 (해시된 비번으로)
→ DB 저장
→ userId 반환 (200)

로그인 (/api/v1/auth/login)

이메일·비번 받기
→ 이메일로 User 찾기 (없으면 401, 계정열거 방지 위해 뭉뚱그린 메시지)
→ 비번 검증 (IPasswordHasher.Verify) (틀리면 401)
→ Access Token 발급 (IJwtTokenGenerator, 15분)
→ Refresh Token 생성 (IRefreshTokenGenerator, 랜덤) + DB 저장 (14일)
→ Access는 body로, Refresh는 httpOnly 쿠키로 (200)

갱신 (/api/v1/auth/refresh)

쿠키에서 Refresh Token 읽기 (없으면 401)
→ DB에서 찾기 (없으면 401)
→ 재사용 탐지: 이미 폐기됐나? → 그렇다면 유저 전체 토큰 무효화 + 거부
→ 만료 검사 (IsActive)
→ 유저 찾기 (IUserRepository.FindByIdAsync)
→ 회전: 옛 Refresh 폐기 (Revoke)
→ 새 Access + 새 Refresh 발급·저장
→ 새 Access는 body, 새 Refresh는 쿠키에 덮어쓰기 (200)

로그아웃 (/api/v1/auth/logout)

쿠키에서 Refresh Token 읽기
→ DB에서 찾아서 폐기 (Revoke) – 없어도 에러 안 냄 (관대하게, 멍등)
→ 쿠키 삭제 (Response.Cookies.Delete)
→ 완료 메시지 (200)

15. 이번에 만든 파일 지도

```
Reservation.Domain/
└─ Entities/RefreshToken.cs      - Refresh 토큰 엔티티 (리치 도메인)

Reservation.Application/
├─ Abstractions/
│  ├── IPasswordHasher.cs        - 해싱 약속
│  ├── IJwtTokenGenerator.cs      - Access 발급 약속
│  ├── IRefreshTokenGenerator.cs  - Refresh 생성 약속
│  └─ IAuthService.cs            - 인증 서비스 약속
│  └─ Repositories/
│     ├── IUserRepository.cs      - User 저장/조회 약속
│     └─ IRefreshTokenRepository.cs - Refresh 저장/조회 약속
├─ Services/AuthService.cs        - 인증 흐름 구현 (오케스트레이션)
└─ Results/AuthResult.cs          - 로그인/갱신 결과 (두 토큰)

Reservation.Infrastructure/
├─ Security/
│  ├── PasswordHasher.cs          - PBKDF2 해싱 구현
│  ├── JwtTokenGenerator.cs        - JWT 발급 구현
│  └─ RefreshTokenGenerator.cs     - 512비트 랜덤 생성
└─ Repositories/
   ├── UserRepository.cs          - User DB 접근
   └─ RefreshTokenRepository.cs    - Refresh DB 접근

Reservation_API/
├─ DTOs/Auth/
│  ├── RegisterUserRequest.cs      - 회원가입 요청
│  ├── LoginUserRequest.cs         - 로그인 요청
│  └─ LoginResponse.cs            - 로그인 응답 (Access만)
├─ Controllers/AuthController.cs   - 4개 엔드포인트 + 쿠키
└─ Program.cs                     - DI 등록 + 인증 미들웨어 + 컨트롤러 매핑
```

16. 한 장 요약 (자기 전에 이것만)

1. HTTP는 **stateless** → 매 요청 신원 증명 필요 → 토큰
2. **JWT** = 서명으로 위조 방지, 서버 저장 없이 검증 (근데 무효화 불가)
3. **딜레마**: 길면 위험, 짧으면 불편 → **Access(짧고 stateless) + Refresh(길고 DB저장=무효화 가능)**
4. **저장**: Access는 메모리, Refresh는 httpOnly 쿠키 (XSS 방어) + SameSite (CSRF 방어)
5. **Rotation + 재사용 탐지**: Refresh 일회용, 폐기된 게 또 오면 탈취로 감지
6. **클린 아키텍처**: 규칙(Application, DB 모름) / 구현(Infrastructure) / 인터페이스로 분리
7. **DI**: 부품을 밖에서 주입, Program.cs에서 조립
8. **4개 엔드포인트**: register / login / refresh / logout — 풀 사이클 완성

작성: Phase 1-5 완료 시점. 다음 단계: 1-F 멀티테넌시 (Global Query Filter로 tenantId 격리).